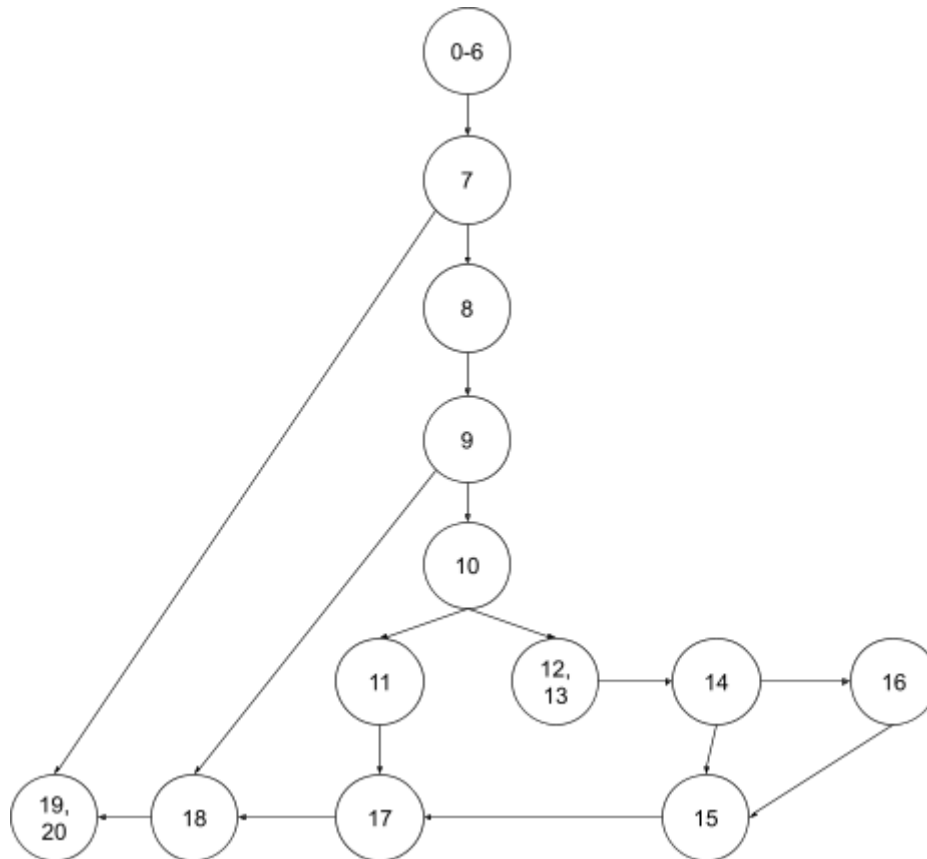


Answer to question no. 1:

a) From slide 1, page 24

b)

i)



ii) The complexity of the code is 5 (decision node +1 = 4+1)

iii) Independent Path:

1) 0-6,7,19,20

2) 0-6,7,8,9,19,20

3) 0-6,7,8,9,10,11,17,18,19,20

4) 0-6,7,8,9,10,12,13,14,16,17,18,19,20

5) 0-6,7,8,9,10,12,13,14,15,16,17,18,19,20

iv) Test cases for each of the independent paths: (Sample)

1) Input: Work: 0, Expected Output: Payment: 0

2) Input: Work: 10, Expected Output: Payment: 40

3) Input: Work: 21, Expected Output: Payment: 38

4) Input: Work: 31, Expected Output: Payment: 90.1

5) Input: Work: 29200, Expected Output: Payment: 2706.3

v) All C-Use Paths for **Payment** variable:

1) 5-6-19

2) 8-9-10-11

3) 8-9-10-12-13

- 4) 8-9-18-19
- 5) 11-17-18-19
- 6) 13-14-15
- 7) 13-14-16-17-18-19
- 8) 15-16-17-18-19

All P-use for **Work** variable:

- 1) 6-7
- 2) 6-7-8-9
- 3) 6-7-8-9-10

- c) Higher level and/or lower level design documents fault, misunderstand/ignore/overthink specific specification

Answer to question no. 2:

- a) Yes

```
s1: if (a > 0 || b > 0){
s2: a++;
s3: } else{
s4: b++;
s5: }
```

100% Multiple-condition coverage requires test cases which cover {a > 0 as true, b > 0 as true}, {a > 0 as true, b > 0 as false}, {a > 0 as false, b > 0 as true}, and {a > 0 as false, b > 0 as false}. If 100% multiple-condition coverage is satisfied then all decisions must be evaluated as both true and false.

- b) Test session number for:

Pairwise integration testing: 13

Neighborhood based: 12-7 =5

- c)

Static Analysis	Dynamic Analysis
The code is not executed.	The code is executed.
Static analysis is performed before code development.	Dynamic Analysis is performed after code development.
E.g; Verification	E.g; Validation

System Testing	User Acceptance Testing
System testing is done to check whether the software or product meets the specified requirements or not.	Acceptance testing is the type of testing which is used to check whether the software meets the customer requirements or not.
System testing is used by testers.	Acceptance testing is used by testers, stakeholders as well as clients.

System testing is both functional and non-functional testing.	Acceptance testing is only functional testing.
System testing is done before acceptance testing.	Acceptance testing is done after system testing.

Answer to question no. 3:

a) **States:**

Idle: At the beginning, without any input

Paid\$5: When \$5 is paid

Paid\$10: When \$10 is paid

Due\$5: When \$5 is paid for coffee

Return\$5: When \$10 is paid for tea

MakeTea: Disposing tea

MakeCoffee: Disposing Coffee

Transactions:

Idle to Paid\$5: 5-Coffee, 5-Tea

Idle to Paid\$10: 10-Coffee, 10-Coffee

Paid\$5 to Due\$5: 5-Coffee

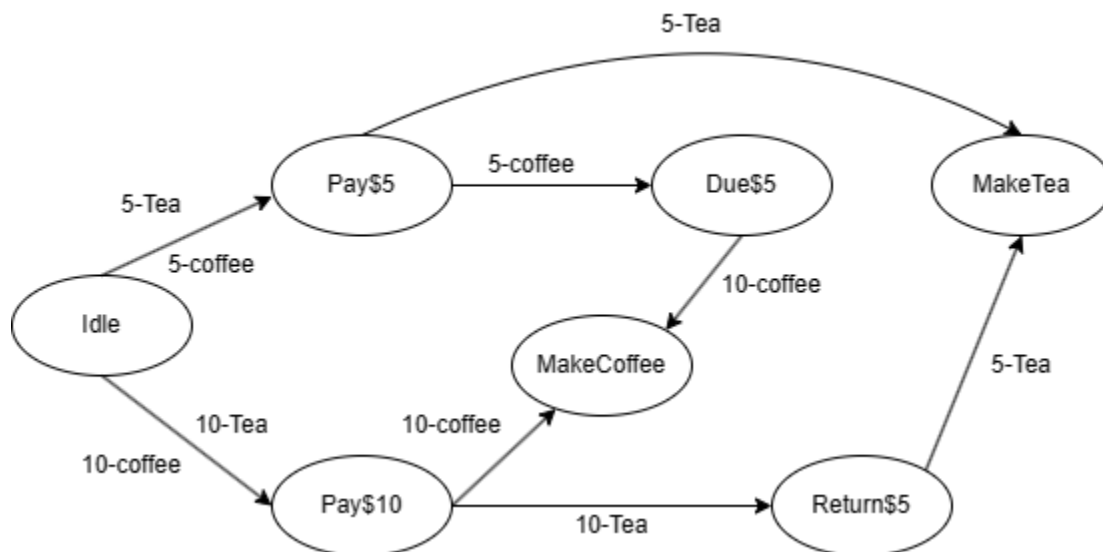
Due\$5 to MakeCoffee: 10-Coffee

Paid\$5 to MakeTea: 5-Tea

Paid\$10 to Return\$5: 10-Tea

Return\$5 to MakeTea: 5-Tea

Paid\$10 to MakeCoffee: 10-Coffee



State Transition Table:

	5-Tea	5-Coffee	10-Tea	10-Coffee
Idle	Pay\$5	Pay\$5	Pay\$10	Pay\$10
Pay\$5	MakeTea	Due\$5	Pay\$5	Pay\$5
Pay\$10	Pay\$10	Pay\$10	Return\$5	MakeCoffee
Due\$5	Due\$5	Due\$5	Due\$5	MakeCoffee
Return\$5	MakeTea	Return\$5	Return\$5	Return\$5
MakeTea	MakeTea	MakeTea	MakeTea	MakeTea
MakeCoffee	MakeCoffee	MakeCoffee	MakeCoffee	MakeCoffee

(Disclaimer: The answer might differ according to your considered states and transitions.)

- b) Slide 5 part 1, page 19-20
- c) Weak Normal Equivalence class testing, Weak Robust Equivalence class testing
It is called the single fault assumption, which states that faults are very rarely the result of the simultaneous occurrence of two or more faults. Both testing techniques assume that it is enough to test each equivalence class once.